

Lab 2. 스테핑 모터 제어

실습 1. GPIO를 이용한 스테핑 모터 구동

- 홈페이지에 제공된 **DSP code**를 다운로드하여 압축을 풀어 사용. 배포된 코드는 **skeleton**에 해당하며, 1상여자 구동방식을 적용하여 0.1초에 1 step씩 무한히 정회전 또는 역회전하는 코드로 배포됨. 각 상에서 일정한 시간의 **delay**를 적용하도록 구현함. 아울러 타이머 인터럽트는 100 kHz로 적용됨. **WaitTFlagCnt()** 함수의 설정값 1은 10usec delay에 해당함.
- Delay time**을 변경하여 모터의 회전 속도를 변경. **Delay time**이 과도하게 짧아서 모터가 회전하지 않는 상태까지 가변.
- 2에서 **delay**가 짧아서 회전하지 않는 상태에서 손으로 모터를 회전시켜보자. 일단 회전하고 나면 회전이 잘 이루어지는 것을 확인.
- 스테핑 모터 구동회로는 내장 **FPGA**에 구현하였으며, 관련 레지스터는 다음과 같음.

IO	Word Offset	Addr. Offset	Read / Write	Description
STEP_PHASE_R	0x030	0x0200_00C0	R/W	Right stepping motor phase
STEP_PHASE_L	0x031	0x0200_00C4	R/W	Left stepping motor phase
Bit[31:8]				Bit[3:0]
N.A.				STPPHA
STPPHA	Stepmotor step phase STPPHA[3:0] = {B, /B, A, /A}			

- 배포된 코드는 **Full Step** 구동으로 작성됨. **Half Step** 구동 방식으로 변경하여 적용하시오.
- Full Step**과 **Half Step** 각각에서 모터 1 회전에 몇 스텝이 필요한지 실험적으로 확인해보시오. (실험에 사용된 스텝 모터의 스텝각은 1.8 deg 임을 고려하여 결과와 비교할 것)

실습 2. 스테핑 모터 구동 API 작성

- 지금 부터는 **Full step** 구동으로 구현.
- 본 과제에서는 시간 지연 함수로 **WaitTFlagCnt()** 함수를 사용. **WaitTFlagCnt()** 함수가 어떻게 동작하는지 코드를 분석해본다. 참고로, 배포된 코드에서 타이머 인터럽트는 100kHz로 적용되었으므로 **WaitTFlagCnt(1e5);** 을 실행하면 1초 delay 적용.
- 스테핑 모터를 정방향 또는 역방향으로 1 스텝 회전하는 함수로 **void OneStepMove(unsigned int dir, unsigned int tDelayCnt)** 를 작성. **dir**이 0이면 정방향, 1이면 역방향으로 회전. (정방향과 역방향은 각자 정의) 현재 인가된 상의 상태를 알아야 다음 상태를 생성할 수 있음에 유의. 모터에 인가하는 상을 변환한 후 **tDelayCnt**에 지정한 횟수 만큼 타이머 인터럽트가 발생할 때까지 대기한 후 함수에서 **return**.
- void StepMoveCV(float angle, float spd)** 함수를 작성. deg 단위의 **angle**에 대해 해당하는 스텝 수만큼 회전하되, 0보다 크면 정방향, 0보다 작으면 역방향으로 회전. **spd**는 회전 스피드에 해당하며, deg/s (degree per sec) 단위로 입력한다. 함수 내부에서 **angle**과 **spd**로부터 회전 방향과 스텝 수, 그리고 스텝간 시간지연을 타이머 인터럽트 주기 수로 환산한 후 1에서 작성한 **OneStepMove**를 호출하여 구현.

실습 3. 속도 프로파일의 적용

1. 앞의 실습 2에서는 고정된 속도로 스테핑 모터를 구동하는 실습을 진행. 하지만, 처음부터 너무 빠른 속도로 구동을 시도할 경우에 탈조가 발생하여 의도한 대로 회전하지 않는 증상을 확인할 수 있음. 즉, `spd` 값을 너무 크게 설정하면 정상 동작하지 않음.
2. 일정 각속도로 속도를 증가시키는 사다리꼴 속도 프로파일을 적용시키는 방법에 대하여 생각하고, 이를 적용하여 함수 `void StepMoveVP(float angle, float maxVel, float accel)` 를 다음의 절차에 따라 작성. `angle`은 양/음수에 따라 회전 방향이 결정되지만, `maxVel`과 `accel`은 절대값으로만 사용.
3. 사다리꼴 속도 프로파일의 결정을 위해서는 가속도, 최대속도, 그리고 회전할 각도 등 3가지로부터 룩업테이블을 작성. 룩업테이블에는 각 스텝간 시간 지연을 타이머 인터럽트 발생 횟수로 저장. 전체 이동 스텝에 대하여 작성할 필요는 없으며, 가속 구간에 대해서만 작성. 이를 담당하는 함수로 `unsigned int MakeVelProfile(float maxVel, float accel)` 를 작성. 룩업테이블은 `unsigned int array type`으로 최대 200개 정도의 `array`를 사용. 함수에서 200개 이상의 시간지연 값을 기록하지 않도록 주의하여야 하며, 가속 구간에서 이동할 스텝 수를 `return`. 가속 구간의 스텝 수는 절대로 200을 넘어서는 안되며, 생성된 룩업테이블을 `MACRO_PRINT`로 출력하여 이상 없음을 반드시 확인. 실제로 잘 튜닝된 스텝 모터 구동에서는 가속 구간의 스텝 수가 일반적인 경우에 최대 50 스텝을 넘어가지 않음.
4. `StepMoveVP`에서는 `MakeVelProfile` 함수를 호출하여 룩업테이블을 생성하고, 생성된 룩업테이블에 저장된 스텝 지연 시간과 가속 구간의 스텝 수를 활용하여 스텝 모터를 회전다. 가속 구간의 스텝 수를 이용하면 전체 이동 스텝 중 현재가 가속, 등속, 감속 구간 중 어디에 속하는지 확인할 수 있음. 가속 구간에서는 매 스텝 이동 마다 룩업테이블을 읽어오는 인덱스를 증가시키고, 등속에서는 계속 같은 위치의 `tDelayCnt` 값을 읽어오며, 감속에서는 인덱스를 감소시키며 읽어옴.
5. 스텝 모터가 1회전하도록 `StepMoveVP` 함수를 호출하되, `maxVel`과 `accel` 값은 너무 크지 않은 값으로 적용하여 구동. 일단 1회전이 정상적으로 회전하면 `maxVel`과 `accel`을 점차 증가시켜서 탈조가 발생하는 `maxVel`과 `accel` 값을 확인. 그 값의 70%에 해당하는 값을 적용하면 안전하게 스텝 모터를 구동할 수 있음. 경우에 따라 기구의 공진 주파수와 상전환 주파수가 일치하면 진동으로 인해 탈조가 발생하기도 하므로 이 경우에 해당하는지 확인 필요.